Combinatorial BLAS 2.0: Scaling Combinatorial Algorithms on Distributed-Memory Systems

Ariful Azad, Oguz Selvitopi, Md Taufique Hussain, John R. Gilbert, and Aydın Buluç

Abstract—Combinatorial algorithms such as those that arise in graph analysis, modeling of discrete systems, bioinformatics, and chemistry, are often hard to parallelize. The Combinatorial BLAS library implements key computational primitives for rapid development of combinatorial algorithms in distributed-memory systems. During the decade since its first introduction, the Combinatorial BLAS library has evolved and expanded significantly. This article details many of the key technical features of Combinatorial BLAS version 2.0, such as communication avoidance, hierarchical parallelism via in-node multithreading, accelerator support via GPU kernels, generalized semiring support, implementations of key data structures and functions, and scalable distributed I/O operations for human-readable files. Our article also presents several rules of thumb for choosing the right data structures and functions in Combinatorial BLAS 2.0, under various common application scenarios.

Index Terms—Sparse matrices, parallel computing, combinatorics, graph theory, communication-avoidance algorithms

1 Introduction

Combinatorial BLAS, or CombBLAS for short, is a distributed-memory library that provides a set of matrix and vector data structures as well as highly-optimized implementations of fundamental operations on and among those data structures. The original purpose of CombBLAS was to provide a proof-of-concept implementation of graph algorithms in the language of linear algebra, demonstrating the feasibility of this approach and the scalability of the resulting implementation. CombBLAS has been used as a benchmark by competing distributed-memory graph libraries. Since launch of the GraphBLAS standardization effort [1], the API development team [2] relied on the design and naming choices made by CombBLAS.

Since its inception a decade ago [3], CombBLAS has been used in a wide variety of distributed data analytics and scientific computing applications. CombBLAS has also evolved heavily to take advantage of the developments in distributed algorithms and architectures. This paper describes the evolution of CombBLAS over the last decade. The introduction surveys the main contributions; the later sections go into their detail.

Avoiding Communication. Communication is the primary bottleneck in scaling data-intensive applications to exascale. Communication-avoiding (CA) algorithms reorganize the computation to reduce communication costs, often asymptotically, and expose more parallelism. CombBLAS was the first library to include a 3D (or 2.5D) sparse matrix-matrix (SpGEMM) multiplication algorithm and since then the algorithm has been expanded to minimize communication under a given (often tight) memory budget. We discuss this integrated CA algorithm in Section 3. Recently, we also integrated a 1.5D sparse times tall-skinny dense matrix multiplication (SpMM) algorithm into CombBLAS.

Hierarchical Parallelism. Exascale computers are going to be based on either accelerators or multi-core CPUs. Original CombBLAS used to run on a so-called "flat-MPI" model where each core was tasked with running an MPI process. With the core counts per compute node increasing from single digits to almost triple digits, a flat MPI model is now known to be unscalable due to increased communication bottlenecks in Network Interface Card (NIC) [4]. Several hierarchical programming models have been proposed where the inter-node communication is handled by either MPI or a different distributed communication library, and the intra-node parallelism is handled via a multithreading platform such as OpenMP. It is also possible to use MPI hierarchically where a smaller MPI communicator is used within a node, an approach known as MPI+MPI.

CombBLAS 2.0 follows the most popular paradigm of using OpenMP parallelism within a node, and MPI for communication across nodes. One reason we avoided a process-based MPI+MPI approach is load imbalance. While CombBLAS avoids most load balance issues by randomly permuting sparse matrices during their assembly, the load imbalance can still hurt the performance if the library runs on 100,000 processes. The use of OpenMP within a node allows CombBLAS to rein in load imbalance since it reduces the number of partitions of a sparse matrix by a factor proportional to the degree of on-node multithreading. Furthermore, popular sparse data structures such as Compressed Sparse Columns (CSC) [5] become wasteful as local matrices become hypersparse [6] due to 2D or 3D decomposition on large numbers of partitions. CombBLAS tames hypersparsity either by specialized data structures such as Doubly-

- *Ariful Azad and Md Taufique Hussain are with the Indiana University, Bloomington, IN 47405 USA. E-mail: azad@iu.edu, mth@indiana.edu.*
- *Oguz Selvitopi and Aydın Buluç are with the Lawrence Berkeley National Laboratory, Berkeley, CA 94720 USA. E-mail: {roselvitopi, abuluc}@lbl.gov.*
- *John R. Gilbert is with the University of California, Santa Barbara, Santa Barbara, CA 93106 USA. E-mail: gilbert@cs.ucsb.edu.*

Manuscript received 25 Feb. 2021; revised 15 June 2021; accepted 16 June 2021. Date of publication 1 July 2021; date of current version 15 Oct. 2021.
(Corresponding author: Aydın Buluç.)
Recommended for acceptance by S. Alam, L. Curfman McInnes, and K. Nakajima.
Digital Object Identifier no. 10.1109/TPDS.2021.3094091

1045-9219 © 2021 IEEE. Personal use is permitted, but republication/redistribution requires IEEE permission.
See https://www.ieee.org/publications/rights/index.html for more information.
Authorized licensed use limited to: University of Maryland College Park. Downloaded on March 17,2024 at 20:35:50 UTC from IEEE Xplore. Restrictions apply.

Compressed Sparse Columns (DCSC) or by increasing in-node multithreading that results in fewer MPI processes (and hence fewer matrix partitions). Virtually all CombBLAS functions are multithreaded within a compute node using the state-of-the-art algorithms, the details of which are given in Section 4.

Accelerator Support. CombBLAS introduces accelerator support for several key operations to address the shift in supercomputer architectures towards exascale. Widely used operations with relatively high arithmetic intensity such as SpGEMM, SpMM, or the multiplication of a sparse matrix with a vector (SpMV) can now effectively make use of accelerators. Tailored algorithms and optimizations for these key operations aim to utilize all resources of a node through a joint execution framework. Details about the accelerator support in CombBLAS are described in Section 5.

User-Defined Operations and Types. Several properties set CombBLAS apart from other distributed matrix libraries. The primary one is the capability to use arbitrary functions and semirings in its operations. The ability to pass user-defined functions and semirings allow CombBLAS to be extremely flexible in terms of implementing a wide range of combinatorial algorithms, including those arising in graph computations [7], [8] as well as other areas such as computational biology [9], [10]. CombBLAS also takes advantage of C++ templates to a high degree, allowing the user to implement previously unspecified operations by just changing the template type of matrix/vector values and the semiring definition. For example, it takes approximately 10 lines of code to perform neighborhood aggregation on vector valued data instead of scalars or to operate on a matrix where each element is another smaller matrix block.

CombBLAS 1.0 had basic semiring support but it was limited in the sense that the elements of two inputs (matrices or vectors) as well as the output had to come from the same set (e.g., $\mathbb{R}$, $\mathbb{Z}$, or $\mathbb{B}$). However, in a significant portion of graph algorithms, those elements come from different sets. For example, the graph can have floating-point weights whereas the vector indices are a pair of integers [7]. CombBLAS 2.0 removes this restriction and allows operations on three different types (two for inputs and one for the output), effectively implementing heterogeneous algebras [11].

Similarly, CombBLAS 1.0 only allowed one type of index, such as `uint32_t` or `uint64_t`. It is often the case that the sparse matrices CombBLAS operates on are too large to be indexed globally with 32-bit integers but they are small enough that the submatrices (or subvectors) assigned to individual processors can be indexed with 32-bit integers. CombBLAS 2.0 allows objects to have two different index types: one for the global indices, say `uint64_t`, and one of the local indices, say `uint32_t`. The storage requirements follow the local index size, while the global index is only used for calculating offsets when communicating with other processors or returning global indices to the user, enabling processing of huge datasets that would otherwise be impossible with a single 64-bit index type.

Parallel I/O on Human-Readable Data. Ability to input and output data efficiently is a fundamental component of any software library that is used for data-intensive computing. Since CombBLAS is designed for problems that are too large to be efficiently handled by a single compute node, the I/O routines also need to be fully parallelized for them to not be a bottleneck. CombBLAS 2.0 introduces parallel I/O support for two human readable text formats, in addition to continuing parallel I/O support for our proprietary binary format. While processing human-readable text files have their overheads compared to binary formats, I/O has never been a bottleneck in CombBLAS because we have fully parallelized our I/O routines. We explain the details of our parallel I/O in Section 6.

Related Work and Availability. The GraphBLAS API [2] is fully implemented by SuiteSparse::GraphBLAS [12] for serial and shared-memory executions. GraphBLAST [13] also closely follows the GraphBLAS API for GPUs but it takes a more pragmatic approach and only implements a subset of the operations commonly needed for graph algorithms. Cyclops Tensor Framework (CTF) [14] is another distributed-memory library that performs matrix computations on arbitrary functions and semirings. With the introduction of the GraphBLAS API, we expect more libraries to have the capability of executing on user-defined functions. The current repository for Combinatorial BLAS is hosted at https://github.com/PASSIONLab/CombBLAS.

2 Technical Background

2.1 Basic Data Structures

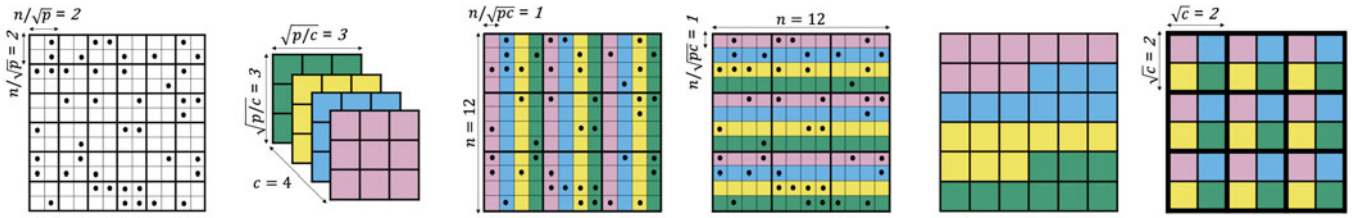
Conceptually at a high level, CombBLAS library has two major data structures: matrices and vectors. Both matrices and vectors have sparse as well as dense versions. Since the primary use case of CombBLAS is data analytics, the overwhelming majority of operations involve at least one sparse matrix. A comprehensive background on using linear algebra for graph and data analytics tasks is beyond the scope of this paper and we refer the interested readers to the book by Kepner and Gilbert [15].

The data structures are either stored in a single address space (henceforth called *local*) or distributed. We explicitly do not make a distinction between serial and multithreaded when we refer to data structures but the methods that operate on objects are naturally distinguished based on threading. The typical end user is expected to interact with CombBLAS via its distributed classes, e.g., `SpParMat` and `FullyDistSpVec`.

The distributed sparse matrix class `SpParMat` is parameterized by the abstract base class `SpMat`, which is a local sparse matrix. Any sparse matrix data structure can derive from `SpMat` as long as it implements the `SpMat` interface, which consists of a few fundamental methods that are needed by CombBLAS. Currently, we have three such classes implemented: `SpDCCols`, `SpCCols`, and `SpTuples`. `SpDCCols` and `SpCCols` are wrappers around DCSC and CSC formats, respectively. `SpTuples` is a wrapper around a C++ std::vector composed of std::tuple entries and is mostly used for interfacing with other libraries and for I/O.

`SpMat` requires its derived classes to implement two iterators that allow power users to implement novel graph and data analysis algorithms in a data structure agnostic manner without going through the CombBLAS primary interface. The first iterator iterates over all nonzero columns of `SpMat`, and the second iterator goes over all the nonzero entries

Authorized licensed use limited to: University of Maryland College Park. Downloaded on March 17,2024 at 20:35:50 UTC from IEEE Xplore. Restrictions apply.



(a) A 12×12 sparse matrix distributed in a 2D 6×6 grid of 36 processes. (b) A 3D grid of 36 processes organized in four 2D 3×3 grids. (c) Partitioning $\mathbf{A}$ into the 3D grid by splitting up the columns. (d) Partitioning $\mathbf{B}$ into the 3D grid by splitting up the rows. (e) Converting a 6×6 grid to a $4\times3\times3$ grid in the regular way. (f) Conversion from 2D to 3D grid using reduced communicators.

Fig. 1. 2D and 3D distribution of a sparse matrix in CombBLAS. Purple, blue, yellow and green colors represent the first, second, third and fourth layers respectively.

within a column of `SpMat`. CombBLAS, therefore, favors column-based data structures for higher performance.

2.2 Data Distribution

The distinction among 1D, 2D, and 3D distributions is whether the underlying process grid is indexed by one, two, or three components. How exactly the matrix is laid out on a given n-dimensional process grid is specific to the underlying distributed object class. CombBLAS does 2D distribution by default for all matrices but it also allows 3D distribution (i.e., distributing into a process grid that is indexed by three components) for its sparse matrices (Section 3.2 and Fig. 1 describe these distributions).

For vectors, CombBLAS supports a 2D decomposition by default, which is superimposed over the default 2D matrix decomposition. If a vector $\mathbf{v} \in \mathbb{S}^n$ is distributed on p processes logically organized on a virtual $p_r \times p_c$ grid, then $\mathbf{v_i}$ is the subvector of length $|\mathbf{v_i}|=\lfloor n/p_r \rfloor$ (except for the last piece $\mathbf{v_{p_r-1}}$ which is of length $n-(p_r-1)\lfloor n/p_r \rfloor$) that is owned collectively by all the processes in the ith process row $P(i,:)$. The subvector $\mathbf{v_i}$ is further partitioned into p_c pieces where each process $P(i,j)$ *exclusively* owns the subvector $\mathbf{v_{ij}}$ of length $\lfloor |\mathbf{v_i}|/p_c \rfloor$ (same remainder rules apply for parts owned by the last process in a process column, i.e., $P(i,p_c-1)$). We emphasize that there is no replication in this vector distribution: to form $\mathbf{v_i}$ on processor $P(i,j)$, an explicit gather operation is needed along process row $P(i,:)$.

CombBLAS supports two types of distributions for storing dense matrices: a 2D distribution like the one used for sparse matrices and the superimposed 2D distribution used for vectors. From matrix point of view, the former distribution splits the dense matrix along both rows and columns (except when p_r=1 or p_c=1), while the latter distribution splits the dense matrix only along the row dimension. These two cases have their use-cases in different scenarios: CombBLAS utilizes the superimposed 2D distribution for tall-skinny dense matrices and true 2D distribution for other dense matrices. All local dense matrices are stored in row-major order. The choice of distribution for dense matrices leads to different algorithms for operations such as sparse matrix times multiple dense vectors (SpMM) that involve dense matrices. For example, in parallel distributed SpMM, the superimposed 2D distribution leads a parallel algorithm in which two dense matrices are communicated. For tall-skinny dense matrices, this is preferable to parallel SpMM algorithms that communicate a sparse and a dense matrix – which is the case for true 2D distribution [16].

2.3 Notation

The key operations in CombBLAS are among matrices of the kind $\mathbf{A} \in \mathbb{S}^{m\times n}$ and vectors of the kind $\mathbf{v} \in \mathbb{S}^n$. $\mathbb{S}$ is the set of the semiring from which the entries of the matrices and vectors are chosen. Both matrices and vectors are distributed on p processors. We use the notation $\mathbf{A}(i,:)$ to refer to the ith row, $\mathbf{A}(:,j)$ to refer to the jth column, and $\mathbf{A}(i,j)$ to refer to the (i,j)th entry of $\mathbf{A}$. Only $nnz(\mathbf{A})$ out of $m \cdot n$ possible entries of a sparse matrix are nonzero where $nnz(\mathbf{A}) \ll m \cdot n$. For notational simplicity in discussing multithreaded algorithms within a node, we denote a local submatrix of $\mathbf{A}$ by $\tilde{\mathbf{A}}$.

3 Reducing Communication

3.1 Communication Costs

While CombBLAS implements many operations, there are only a handful of *key operations* that directly affect the application runtime. Those key operations are computing the product of two sparse matrices (SpGEMM), sparse-matrix dense-vector multiplication (SpMV) and its multi-vector generalization (SpMM), sparse-matrix sparse-vector multiplication (SpMSpV), and vector extraction and assignment (VecAssign). We use the syntax $\mathbf{C} = \mathbf{AB}$ for SpGEMM, $\mathbf{y} = \mathbf{Ax}$ for SpMV and SpMSpV, and $\mathbf{Y} = \mathbf{AX}$ for SpMM. Here, $\mathbf{x}$ and $\mathbf{y}$ are (dense or sparse) vectors. $\mathbf{A} \in \mathbb{S}^{m\times n}$, $\mathbf{B} \in \mathbb{S}^{n\times k}$, and $\mathbf{C} \in \mathbb{S}^{m\times k}$ are sparse matrices. $\mathbf{X} \in \mathbb{S}^{n\times k}$ and $\mathbf{Y} \in \mathbb{S}^{m\times k}$ are dense matrices.

To develop an analytical framework for key CombBLAS operations, we assume that the data and computations are balanced across p processes organized in a $\sqrt{p}\times\sqrt{p}$ process grid P_{2D}, meaning that each process stores approximately $nnz(\mathbf{A})/p$ nonzeros of a matrix $\mathbf{A}$ and n/p entries of a vector of length n. We emphasize that this assumption is artificial and just made for simplifying the complexity analysis. CombBLAS allows the user to randomly permute sparse matrices during matrix assembly, using which load balance can be achieved with high probability [17].

Given this balanced setting, Table 1 shows the computation and communication complexity of these operations. We see that key CombBLAS operations have low computational intensity, and their performances are bound by memory-to-processor and inter-processor communication costs.

Authorized licensed use limited to: University of Maryland College Park. Downloaded on March 17,2024 at 20:35:50 UTC from IEEE Xplore. Restrictions apply.

TABLE 1
Communication Complexity of Key CombBLAS Operations When Matrices and Vectors are Distributed on a 2D $\sqrt{p}\times\sqrt{p}$ Process Grid

Operation	Computation	Latency	Bandwidth
SpGEMM	$O(\text{flops}/p)$	$O(\sqrt{p}\lg p)$	$O(nnz(\mathbf{A}+\mathbf{B})/\sqrt{p})$
SpMM	$O(k\cdot nnz/p)$	$O(\lg p)$	$O(k(m+n)/\sqrt{p})$
SpMV	$O(nnz/p)$	$O(\sqrt{p})$	$O(n\lg p/\sqrt{p})$
SpMSpV	$O(nnz\cdot f/p)$	$O(\sqrt{p})$	$O((nf+ng)/\sqrt{p})$
VecAssign	$O(n/p)$	$O(p)$	$O(n/p)$

flops *denotes the number of nonzero arithmetic operations required when computing SpGEMM between* **A** *and* **B**. *For SpMSpV, the density of the input vector is* f *and the unreduced output vector is* g.

Table 1 also shows that communication becomes increasingly dominant as we move down the table. Furthermore, communication latency becomes increasingly significant relative to bandwidth costs as we move from matrix to vector operations. This theoretical analysis is also observed in graph algorithms implemented on top of CombBLAS. Thus, different algorithms need different strategies to minimize communication depending on the relative importance of computation, bandwidth and latency costs.

Reducing communication cost can be achieved either algorithmically, which provides asymptotic improvements, or via methods that change implementation details (in particular, managing fine-grained communication) to reduce costs without changing the asymptotic complexity. The former only applies to operations that have arithmetic intensity bounded above 1. Hence, we apply asymptotically better communication-avoiding methods to SpGEMM and various fine-grained communication schemes to other operations.

3.2 Communication-Avoiding SpGEMM

In earlier versions of CombBLAS, distributed SpGEMM was based on the 2D SUMMA algorithm [18] adapted for sparse matrices [19]. In 2D Sparse SUMMA, input matrices **A** and **B** and the output matrix **C** are distributed on a $\sqrt{p}\times\sqrt{p}$ process grid. The simplest variant of 2D Sparse SUMMA operates in $\sqrt{p}$ stages. In the ith stage, each process in the ith process column broadcasts their local piece of **A** horizontally (along the process row) and each process in the ith process row broadcasts their local piece of **B** vertically (along the process column). The received submatrices are locally multiplied by each process using a multithreaded SpGEMM algorithm (Section 4). After $\sqrt{p}$ stages, partial results from all stages are merged to obtain the final result.

The costs of broadcasting input matrices in 2D SUMMA quickly become a performance bottleneck at extreme scale [19], [20]. To alleviate this bottleneck, CombBLAS 2.0 includes CA SpGEMM algorithms, following the success of CA algorithms in dense linear algebra [21]. Our CA SpGEMM algorithms distribute matrices on a $c\times\sqrt{p/c}\times\sqrt{p/c}$ process grid, where c denotes the number of layers in the third dimension. Fig. 1b shows an example of a 3D process grid with four layers, where each layer is equivalent to a $\sqrt{p/c}\times\sqrt{p/c}$ 2D process grid. To facilitate 2D SUMMA algorithm in each layer, we split **A** along the column and **B** along the row into c pieces and then distribute different pieces to different layers as illustrated in Figs. 1c and 1d.

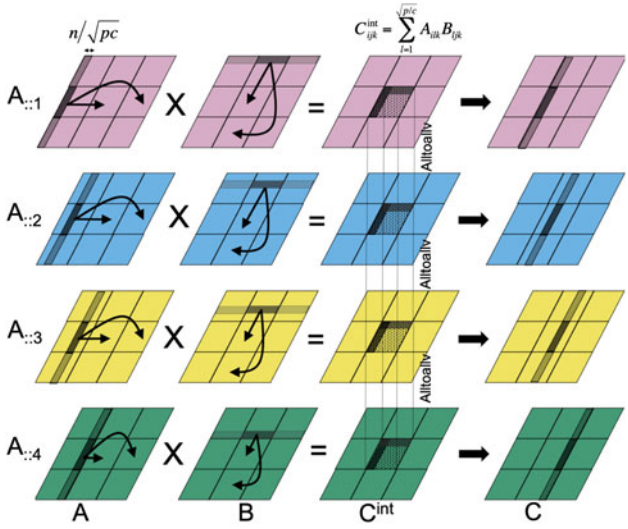


Fig. 2. Execution of the communication avoiding SpGEMM for multiplying sparse matrices $\mathbf{A}$ and $\mathbf{B}$ to get $\mathbf{C}$ on a $c\times\sqrt{p/c}\times\sqrt{p/c}$ process grid. Shown operations are involved to generate the local portion of $\mathbf{C}$ only for the processes at the second row and the second column of each layer during the first stage of the algorithm.

After input matrices are distributed on a 3D process grid, each layer runs an instance of the 2D SUMMA algorithm to obtain intermediate per-layer results $\mathbf{C}^{int}$ as shown in Fig. 2. Here, each layer broadcasts submatrices of **A** along the process rows and submatrices of **B** along the process columns on the 2D grid represented by the layer. Since these broadcasts materialize on a smaller (by a factor of $\sqrt{c}$) communicator, their costs are reduced at extreme scale [20]. After each layer completes their 2D multiplications, the partial results are communicated across layers via an Alltoall communication. We form the final result **C** by merging pieces received from all layers. Since **A** and **B** are distributed differently on the 3D grid, we distribute **C** like **A** (as shown Fig. 2).

Guideline on Selecting the Number of Layers (c). Unlike dense CA algorithms [21] that replicate input matrices to reduce communication, our CA SpGEMM splits input matrices and does not require any extra memory for inputs with increasing number of layers. Generally, the time required to broadcast **A** and **B** decreases as we increase c (e.g., we could completely eliminate broadcasts by using an $p\times 1\times 1$ grid). However, as c increases, the costs of inter-layer Alltoall communication and the final merging also increase. Furthermore, the memory required to store intermediate results increases with increasing number of layers. Therefore, it is challenging to find the optimum c as it depends on the tradeoff between broadcasts and inter-layer Alltoall communication costs, as well as the available memory. Our general guideline is to select c with $c\leq\sqrt[3]{p}$ so that inter-layer Alltoall does not dominate intra-layer broadcasts.

Conversion Between 2D and 3D Distributions. At present, CombBLAS performs I/O only with 2D matrices (Section 6). In order to use CA SpGEMM algorithms, we convert matrices from a $\sqrt{p}\times\sqrt{p}$ process grid to a $c\times\sqrt{p/c}\times\sqrt{p/c}$ grid. Figs. 1a and 1b show an example how a 12×12 sparse matrix is converted from a 6×6 grid to a $4\times3\times3$ grid. CombBLAS provides two ways to create 3D matrices from 2D matrices. Fig. 1e shows how the conversion is done in the regular way where

Authorized licensed use limited to: University of Maryland College Park. Downloaded on March 17,2024 at 20:35:50 UTC from IEEE Xplore. Restrictions apply.

processes on a 2D grid are numbered in the row-major order and we place p/c processes numbered $\{i, i+1, \ldots, i+p/c-1\}$ into the ith layer. This conversion redistributes a 2D matrix on a 3D grid using an Alltoallv operation among all processes. In the second approach, we reinterpret the whole 2D grid as a $\sqrt{p/c} \times \sqrt{p/c}$ supergrid (shown with thick lines in Fig. 1f) where each cell of the supergrid has a $\sqrt{c} \times \sqrt{c}$ subgrid of the 2D grid. Thus, each supergrid cell corresponds to one cell in each layer of the 3D grid. We then assign each cell of a subgrid to the corresponding cell in each layer of the 3D grid as shown in Fig. 1f. Here, p/c Alltoallv calls run in parallel with each Alltoallv involving c processes. By operating on a smaller communicator, the latter approach reduces the communication cost at extreme scale. For example, when converting the Metaclust50 matrix from a 64×64 grid to a $16 \times 16 \times 16$ grid on 256 nodes, the second approach takes 14 seconds whereas the first approach takes 18 seconds.

3.3 Fine-Grained Communication-Reduction Schemes

For operations with lower arithmetic intensities, CombBLAS provides several other communication-reduction schemes that are often tailored for different applications. We broadly group them into three classes and discuss them in relation to their corresponding applications.

Vector Replications to Avoid Asynchronous Communication in Graph Matching Algorithms. CombBLAS powers distributed-memory maximal cardinality matching [22], maximum cardinality matching [7] and heavy-weight perfect matching [17] algorithms for bipartite graphs. All of these algorithms iteratively match vertices between two parts of a bipartite graph, where endpoints of currently matched edges are stored in two dense vectors (instances of `FullyDistVec`). In iterative matching algorithms, these dense vectors are accessed irregularly to discover alternating paths (paths with alternating matched and unmatched edges) and then updated when the matching is improved. However, `FullyDistVec` does not perform well for such irregular and asynchronous accesses or updates because it distributes all n entries of the vector uniformly among p processes. To alleviate this problem, we replicated dense vectors within a process row and a process column so that each process stores $n/\sqrt{p}$ entries of the vector as opposed to n/p entries stored in the default distribution. Such a vector replication scheme eliminates fine-grained communication among processes and promotes updates in the bulk synchronous fashion. Since the vector is replicated $\sqrt{p}$ times, the memory overhead is significant only when $\sqrt{p} \gg d$, where d is the average degree of the graph.

One-Sided Communication. Even though CombBLAS performs the best with bulk-synchronous parallel executions, it also utilizes asynchronous one-sided communication if needed by certain algorithms. For example, it is inefficient to traverse long paths level-by-level in the bulk-synchronous fashion because of limited parallelism and high synchronization overheads. For this case, we asynchronously traverse long paths using special hardware features such as Remote Memory Access operations (e.g., one-sided put and get operations in MPI). Currently, this approach is only implemented in maximum-cardinality matching algorithms [7], but it could potentially be used for SpMSpV as well.

Collective Communication on Reduced Communicators. Irregular operations performed solely on sparse/dense vectors are harder to scale because of the performance bottleneck arisen from the communication latency that increases linearly with p (see Table 1). This problem exacerbates for iterative algorithms with skewed communication patterns where the data sent and received by processors changes dramatically over iterations. For example, different iterations of the distributed Awerbuch-Shiloach algorithm [23] for finding connected components often process drastically different numbers of vertices. This results in highly-skewed communication where different processes exchange different volumes of data. In the presence of such skewed communication, vector assignment or extract operation implemented with the Alltoall collective executed over the entire process grid does not perform well in practice. CombBLAS alleviates this problem by providing assign/extract operations in time proportional (or as close to possible) to the size of the updated or extracted region. To this end, we identify a small set of processes that send or receive more than 90 percent of the messages and process them separately with broadcast or reduction operations. The rest of the processes then create a reduced sub-communicator where another Alltoall takes place. Currently, this approach is implemented only for vector operations inside the connected component algorithm. However, it can be easily extended to SpMSpV and other vector operations as well.

4 Node-Level Multithreading

Hybrid parallelization using OpenMP/pthread for in-node computation and MPI for inter-node communication has become the dominant programming model for applications running on modern supercomputers. This hybrid parallelization comes with two apparent benefits as it avoids data replication among threads within a process and reduces the number of messages exchanged among processes. For various sparse computations in CombBLAS, the hybrid parallelism provides three other crucial benefits. If the number of processes reduces from in-node multithreading, (1) the sparse SUMMA algorithm for SpGEMM may need less memory to store intermediate results, (2) the communication cost of latency-bound operations such as SpMSpV reduces, and (3) irregular pointer chasing operations such as vector assign become less expensive. Hence, CombBLAS 2.0 includes multithreaded algorithms for almost all operations. For SpGEMM, SpMV, and SpMSpV, we developed multiple algorithms because different algorithms perform better for matrices with different sparsity patterns. SpMM has only one variant due to its lower popularity among those operations. Table 2 summarizes algorithmic options available for various operations in CombBLAS 2.0. All multithreaded algorithms are implemented using OpenMP.

4.1 Multithreaded SpGEMM

In-node SpGEMM in CombBLAS is parallelized based on Gustavson's algorithm [26] that constructs columns of the output matrix in parallel. To compute the jth column $\tilde{\mathbf{C}}(:, j)$ of the output, Gustavson's algorithm merges columns of $\tilde{\mathbf{A}}$ corresponding to nonzeros in $\tilde{\mathbf{B}}(:, j)$. CombBLAS 2.0 provided one of the first multithreaded SpGEMM algorithms

Authorized licensed use limited to: University of Maryland College Park. Downloaded on March 17,2024 at 20:35:50 UTC from IEEE Xplore. Restrictions apply.

TABLE 2
Multithreaded Algorithms Implemented in CombBLAS 2.0

Operation	Work per thread	Algorithms
SpGEMM [20], [24]	a subset of columns in the output matrix	column-by-column using heap, hash table, or hybrid heap-hash
SpMM	a subset of columns in the input matrix	
SpMV	a subset of columns or rows in the input matrix	inner product
SpMSpV [25]	a subset of nonzeros in the input vector	merging using SPA, heap, sorting, or binning

based on heap [20] and hash table [24] data structures. SpGEMM implementations in CombBLAS operate in three phases. (1) Estimate the total flops needed by the entire multiplication and then divide columns of the output across threads so that each thread performs approximately equal work. The complexity of this step is $O(nnz(\tilde{\mathbf{B}}))$ when both $\tilde{\mathbf{A}}$ and $\tilde{\mathbf{B}}$ are stored in CSC or DCSC formats. (2) A symbolic phase estimates the required memory for the output. This step uses the heap or hash table data structure to compute the pattern of $\tilde{\mathbf{C}}$, and its complexity is $O(\text{flops})$. (3) Finally, the computational phase performs the actual multiplication using the heap or hash table data structure, which has the same $O(\text{flops})$ complexity as the previous stage. We refer to the SpGEMM that uses heap and hash data structures as heap-SpGEMM and hash-SpGEMM, respectively

The heap-SpGEMM algorithm usually performs better than hash-SpGEMM when the SpGEMM has a low compression ratio (the ratio of flops to nonzeros in the output). By contrast, hash-SpGEMM excels when the SpGEMM has a high compression ratio [24]. We also implemented a hybrid hash-heap algorithm that uses either a heap or a hash table to form a column of $\tilde{\mathbf{C}}$ depending on the compression ratio of the individual column. This fine-grained decision helps when matrices have skewed and unpredictable nonzero distribution.

4.2 Multithreaded SpMV

CombBLAS 2.0 includes two variants of multithreaded SpMV. Both of them access the matrix column-by-column due to the DCSC and CSC formats used for storing local matrices. In the first variant, the input matrix $\tilde{\mathbf{A}}$ is partitioned row-wise into t parts where each part $\tilde{\mathbf{A}}_i$ with n/t rows is stored in the DCSC or CSC format. Then, each thread multiplies one part $\tilde{\mathbf{A}}_i$ of $\tilde{\mathbf{A}}$ by the input vector and writes to a part of the output vector corresponding to the rows in $\tilde{\mathbf{A}}_i$. In the second variant, $\tilde{\mathbf{A}}$ is partitioned column-wise into t parts where each part $\tilde{\mathbf{A}}_i$ has n/t columns. Then, each thread multiplies $\tilde{\mathbf{A}}_i$ by a part of the input vector corresponding to the columns in $\tilde{\mathbf{A}}_i$ and writes the output to a thread-private output vector y_i. At the end of the multiplication, all thread-private outputs are accumulated to form the final vector in parallel by all threads.

Both row- and column-partitioned algorithms have their advantages and disadvantages. The row-partitioned SpMV does not require extra memory to store partial results and enjoys better locality in accessing the output vector, but the entire input vector is accessed by all threads. Furthermore, row-wise partitioning of DCSC and CSC matrices incurs significant overhead that can only be mitigated in interactive algorithms. In contrast, the column-partitioned SpMV does not need a partitioning step and each thread accesses only a part of the input vector, but it needs extra thread-private memory to store partial results. While users can use either of these algorithms, the row-partitioned SpMV is recommended in iterative algorithms and the column-partitioned SpMV is recommended for non-iterative algorithms.

4.3 Multithreaded SpMSpV

Given a local sparse matrix $\tilde{\mathbf{A}}$ and a local sparse vector $\mathbf{x}$ with f nonzeros, an SpMSpV computes a potentially sparse vector $\mathbf{y} \leftarrow \tilde{\mathbf{A}}\mathbf{x}$. Similar to SpMV, the implementations of SpMSpV in CombBLAS rely on row-wise and column-wise partitioning of the matrix by splitting it into $n/t \times n$ and $n \times n/t$ submatrices for row-wise and column-wise partitions, respectively. For both partitioning schemes, we implemented several multithreaded SpMSpV algorithms that use different strategies to merge columns of the input matrix corresponding to nonzero entries of the input vector. For the row-partitioned case, CombBLAS 2.0 uses heap (SpMSpV-Heap) and SPA (SpMSpV-SPA) to merge columns within each partition. By contrast, the column-partitioned algorithms use buckets for merging (SpMSpV-Bucket).

To understand algorithmic variants of SpMSpV, let d be the average nnz per column of $\tilde{\mathbf{A}}$ and t be the number of threads. Then, the asymptotic lower bound of SpMSpV is $\Omega(df)$. Each thread in a row-partitioned SpMSpV needs to access the entire input vector, giving us the parallel complexity of $O(f + df/t)$. This results in work-inefficient algorithms when $t > d$. By contrast, a column-partitioned algorithm partitions $\mathbf{x}$ among threads, but needs synchronization when updating the output. To alleviate thread synchronizations, we used intermediate buckets to block $\mathbf{y}$ along the rows [25] (also known as propagation blocking [27]). Section 4.5 shows that different SpMSpV algorithms excel for different sparsity patterns of matrices and vectors.

4.4 Multithreaded SpMM

CombBLAS 2.0 includes only one variant for multithreaded SpMM, because it not as frequently used as the operations discussed so far. The multithreaded algorithm used for SpMM is similar to the ones used for SpMV and SpMSpV where the sparse matrix is partitioned column-wise. The dense matrices are stored in row-major order. These choices result in the preference of reuse on the input dense matrix. The overhead due to row-wise partitioning of the sparse matrix may be more readily justified in SpMM than in SpMV or SpMSpV, prompting development of such alternative implementations in future CombBLAS releases.

Authorized licensed use limited to: University of Maryland College Park. Downloaded on March 17,2024 at 20:35:50 UTC from IEEE Xplore. Restrictions apply.

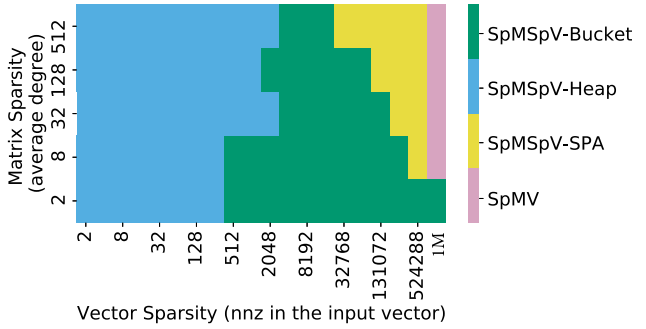


Fig. 3. Multiplying an 1M×1M R-MAT sparse matrix by a sparse vector. We show best performing SpMSpV or SpMV algorithms for different matrix and vector sparsity patterns. Experiments were run on a single node of the Cori Haswell partition with 32 cores.

4.5 Performance of SpMSpV and SpMV Algorithms With Respect to Matrix and Vector Sparsity

As discussed above, CombBLAS 2.0 includes a rich collection of multithreaded SpMSpV and SpMV algorithms. Different algorithms excel with different sparsity patterns of the input matrix and the vector. To provide a guideline for users, we used a $1\text{M} \times 1\text{M}$ randomly-generated R-MAT matrix [28], which is a synthetic graph generator used by the Graph500 benchmark. We used the same parameters as the Graph500 benchmark. We multiplied the R-MAT matrix by a vector on a 32-core Intel Haswell processor. We change the sparsity of the matrix by changing the average number of nonzeros per column in the matrix generator. We also control the sparsity of the input vector by filling it with nonzeros at uniformly random indices. Fig. 3 summarizes the best performing algorithm for different matrix and vector sparsity patterns. We observe that SpMSpV-Heap performs the best with vectors densities less than 0.5 percent, SpMSpV-Bucket performs the best with vectors densities greater than 0.5 percent and less than 10 percent, and SpMSpV-SPA performs the best with denser matrix and vector. Interestingly, SpMSpV-SPA and SpMSpV-Bucket perform similarly to or better than our SpMV implementations even when the vector is 50 percent dense. While there is room to improve the performance of our SpMV implementation, these results suggest that our SpMSpV implementations attain good performance for almost all sparsity levels in the input vector.

5 GPU Acceleration

Operation Support. CombBLAS 2.0 supports NVIDIA GPU accelerators for relatively more compute-intensive and important operations. The supported three such operations are SpGEMM, SpMM, and SpMV. Although the computations on sparse matrices are often ill-suited for accelerators due to their low computational density and irregular memory access patterns, the cuSPARSE library[1] often provides satisfactory performance compared to its counterpart sparse libraries on CPUs, such as MKL, even for SpMV, which has the lowest arithmetic intensity among the three operations. A relatively modest speedup of $3\times$ over the CPU often translates into valuable computational improvement and justifies the overhead of copying

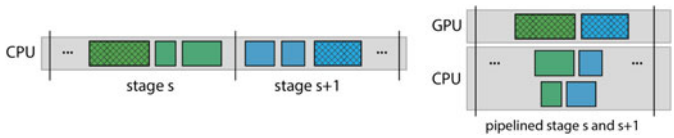


Fig. 4. CPU-GPU joint execution of multi-stage parallel algorithms. The rectangles illustrate various tasks and the patterned rectangles illustrate local computations offloaded to GPU. Any task between two local computations is handled by the host and overlapped with GPU computations.

data back and forth between the host and the device. The SpGEMM and SpMM operations offer even greater promise for relying on accelerators. Moreover, recent works [29], [30] in this area report $2{-}4\times$ speedups over cuSPARSE, making GPUs even more viable for these operations if they are adapted. Based on these arguments, CombBLAS 2.0 provides support for these three operations and certain tailored optimizations. We next discuss how CombBLAS 2.0 handles these operations.

The above-mentioned operations are rarely used alone and they are often part of a larger execution framework that possibly contains other sparse matrix and vector operations. Since such operations in CombBLAS 2.0 are only realized for CPUs and a big proportion of them would make little sense to offload to GPUs, the matrix and vector objects in GPU-enabled CombBLAS 2.0 still dwell in the host memory. One main reason for this choice is related to the nature of the graph algorithms. Most of the graph algorithms require custom binary operators, semirings, and/or custom vector/matrix elements. Support for such customization on GPUs is nearly nonexistent. There are efforts such as cusp,[2] but the support is limited and the library is poorly maintained and occasionally crashes. Hence, CombBLAS 2.0 assumes that its objects mainly live on the host memory and utilize accelerators on a need basis where there is a promise of computational improvement. It has the capability for different processes in the SpGEMM, SpMM, or SpMV to utilize CPU or GPU based on the flops of the target operation. Note that the GPUs are only utilized for the arithmetic semiring. In the existence of non-built-in types for vector/elements or other semirings, CombBLAS 2.0 falls back to CPUs.

Coordination. The distributed algorithms for SpGEMM, SpMM, and SpMV can be distinguished into two according to whether the local computations are split across multiple stages that are separated by various communication operations and other tasks. The parallel algorithms that split computations include the Sparse SUMMA algorithm for SpGEMM and all the SpMM algorithms where the dense matrices are 2D partitioned. The algorithms for SpMV and the SpMM algorithms in which the dense matrices are 1D partitioned do not split computations. For the former case, we adopt a strategy where the device is responsible only for the local computations and the host is responsible for the rest of the tasks. In this direction, we aim to overlap the local computations on GPU in stage s with CPU tasks in stage s or $s+1$ (Fig. 4). Moreover, we distribute CPU tasks that do not depend on each other among the CPU cores of the node. This approach results in pipelined versions of these multi-stage parallel algorithms where the host and the device share work.

1. https://docs.nvidia.com/cuda/cusparse/index.html
2. https://github.com/cusplibrary

Authorized licensed use limited to: University of Maryland College Park. Downloaded on March 17,2024 at 20:35:50 UTC from IEEE Xplore. Restrictions apply.

CombBLAS 2.0 can utilize multiple devices on a node via node-level parallelism. The work distribution among multiple devices on a node depends on the operation. For SpGEMM, among the two local input matrices to be multiplied, the first one is replicated among all devices and the second input matrix is partitioned column-wise among the GPUs. This partitioning conforms to the sparse matrix storage formats and aims to assign roughly equal number of nonzeros to each device. Each device computes a column stripe of the output matrix, which is concatenated with other stripes at the end. For SpMM and SpMV, the sparse matrix is column-wise partitioned in a load-balanced manner among the GPUs along with a conformal distribution of the required input dense matrix/vector. Each device computes partial results for the output matrix/vector which are then reduced by the host. CombBLAS 2.0 uses threads on the host side to manage GPU coordination and any necessary pre-/post-processing.

Optimizations. Among the three supported operations on GPUs, CombBLAS 2.0 has comprehensive optimization for SpGEMM. The ubiquitous nature of this operation together with its high-level complexity necessitate algorithmic techniques for efficient execution on systems with accelerators. For this operation, CombBLAS 2.0 makes use of multiple SpGEMM libraries [29], [31], [32] on GPUs by selecting the most appropriate library according to factors such as compression factor and flops. Other optimizations include faster memory requirement estimation that involves approximate algorithms and a binary merge scheme that spreads out the computations related to merging of the partial results across stages of the SUMMA algorithm. A recent work [33] contains more information about these optimizations.

6 High-Performance Parallel I/O

I/O has increasingly become a bottleneck on exascale systems. The graph community does not have a commonly accepted binary exchange format. CombBLAS defined its binary format since version 1.0 and provided conversion routines. In addition to continuing the support for binary I/O, CombBLAS 2.0 introduces support for parallel I/O on two types of human-readable text file formats.

First is the Matrix Market format of NIST (i.e., .mtx files) [34]. The Coordinate Format of Matrix Market Exchange Format is composed of a header telling whether the matrix is symmetric and the type of nonzero values (*real*, *integer*, *pattern*), followed by a tab separated line giving row (M), column (N), and nonzero counts (nnz). The body of the text has as many lines as the number of nonzeros in the matrix, with each line listing the row id, column id, and value of one nonzero. The row and column ids are integers in range $\{1, \ldots, M\}$ and $\{1, \ldots, N\}$, respectively.

The second is the label format of the original Markov Clustering implementation [35]. This format is similar to the matrix market format except that (a) the header is not present, and (b) the row and column ids can be arbitrary strings. This label format is convenient because it does not require row and column ids to be in consecutive $\{0, \ldots, M-1\}$ and $\{0, \ldots, N-1\}$ ranges. It can handle scattered integers in a wide range by treating them as arbitrary labels. The labels do not even need be integers, e.g., the reader can handle labels that can represent DNA or protein sequences.

The input reader for the Matrix Market case (`ParallelReadMM` function) is relatively straightforward where processor $p_i \in P$ moves its cursor to location $filesize \cdot i/|P|$ in the file. It then fast forwards until it sees a newline to start reading. Conversely, if a process' share of the input ends in the middle of a line, that process is responsible for reading the line until completion. The actual reading happens using MPI I/O collective routines into an in-memory byte stream, which is then parsed locally in each processor. In order to write a Matrix Market file, each process simply writes its set of local nonzeros into a byte stream. The first process (p_o) creates the file and writes the header information. The writing of the body of the .mtx file happens by writing the byte streams into disk using collective MPI I/O operations. This ensures a human readable text output that is also written in parallel.

The case of the label format is more complicated. The file reader needs to internally map arbitrary labels into a consecutive range of integers. This is accomplished by a two pass algorithm in our implementation that is called `ReadGeneralizedTuples`. In the first pass, row (and column) ids are hashed randomly into the range $\{0, \ldots, max\}$. That range itself is partitioned into $|P|$ buckets, one per processor. Then an *all-to-all exchange* sends the row (and column) ids as well as their hashed values to their corresponding processes determined by the bucketing based on hash values. The recipients remove duplicates using a local set data structure. The sizes of the resulting local sets are used to find starting id offsets in each process via an *exclusive scan*, using which each process gives a unique consecutive id to the rows and columns. Each recipient also keeps track of the data sent in this *all-to-all exchange* so it can send back the new ids, which are in the ranges $\{0, \ldots, M-1\}$ and $\{0, \ldots, N-1\}$, back to each sender. The original senders can now build a string-to-integer map that they can use to relabel their arbitrary identifiers. The two-pass algorithm does not need to go back to disk for the second pass if there is enough memory to cache the first read, but our default implementation re-reads from the disk for a streaming relabeling operation that is more memory frugal. For a file that contains $O(nnz)$ nonzeros, the per-process memory consumption of `ReadGeneralizedTuples` is $O(nnz/|P|)$ in expectation. The input is already deterministically partitioned into $|P|$ equal parts and the random hash-based assignment ensures load balance in expectation.

Upon completion, `ReadGeneralizedTuples` returns two objects: (1) a CombBLAS compliant distributed sparse matrix object, and (2) a CombBLAS compliant distributed vector that maps the newly created integer labels $\{0, \ldots, M-1\}$ and $\{0, \ldots, N-1\}$ into their original string labels so that the program can convert the internal labels back into their original labels for subsequent processing or while writing the output. A crucial positive side effect of the `ReadGeneralizedTuples` function is that it automatically permutes row and column ids randomly during the relabeling, ensuring load balance of CombBLAS operations that use the resulting distributed sparse matrix. For this reason alone, one can use the `ReadGeneralizedTuples` function in lieu of the `ParallelReadMM` function if the input is known to be severely load imbalanced. In such cases, reading the input into a distributed sparse matrix `ParallelReadMM` and subsequently permuting it within CombBLAS for load balance might not be feasible,

Authorized licensed use limited to: University of Maryland College Park. Downloaded on March 17,2024 at 20:35:50 UTC from IEEE Xplore. Restrictions apply.

TABLE 3
Overview of the Evaluation Platform

	Cori-KNL	Cori-Haswell	Summit
Processor	Intel Xeon Phi 7250	Intel Xeon E5-2698	IBM Power9
Accelerator	–	–	NVIDIA Volta V100 (6 per node)
Cores/node	68	32	42
Hyper-threads/core	4	2	4
Memory/node	112GB	128GB	512GB Host 96GB HBM
Total nodes	9,668	2,388	4,608
Total memory	1.09 PB	298.5 TB	10 PB
Interconnect	Cray Aries with Dragonfly topology		Non-blocking Fat tree
Compiler	Intel icpc 19.0.3 with -O3		GNU gcc 8.1.1 with -O3 NVIDIA nvcc 11.0.3
MPI Vendor	Cray-MPICH 7.7.10		Spectrum MPI 10.3

TABLE 4
Statistics About Test Matrices Used in Our Experiments

Matrix (**A**)	rows	columns	$nnz(\mathbf{A})$	$nnz(\mathbf{C})$	flops
Virus	0.22M	0.22M	4.58M	7.49M	366M
Archaea	1.6M	1.6M	204M	–	–
uk-2002	19M	19M	298M	–	–
Eukarya	3M	3M	360M	2B	134B
arabic-2005	23M	23M	640M	–	–
Isolates1	8.7M	8.7M	1B	10.7B	386B
it-2004	41M	41M	1.2B	–	–
Isolates2	17.5M	17.5M	4.2B	49B	3T
Friendster	66M	66M	3.6B	1T	1.4T
Metaclust50	282M	282M	37B	1T	92T
Hyperlink	3.27B	3.27B	124.9B	–	–

For SpGEMM experiments, we show flops *and nonzeros in the output. M: million, B: billion and T: trillion.*

because the load imbalance can be high enough for some process to run out of local memory before `ParallelReadMM` finishes.

7 Experimental Results

Over the past decade, CombBLAS made significant progress in (a) developing new algorithms for sparse-matrix primitives [7], (b) implementing algorithms to extract high-performance from heterogeneous distributed systems with CPUs and GPUs [33], (c) demonstrating extreme-scalability using communication-avoiding algorithms that scale to the limit of supercomputers [36], [37], and (d) providing customized functionality for several high-impact applications in computational biology [9], [38] and scientific computing [17]. While many of these progresses have already been published separately, we show the overall impact of moving CombBLAS 1.0 to CombBLAS 2.0 and demonstrate how CombBLAS 2.0 made important progress toward exascale.

7.1 Experimental Settings

We evaluate the performance of CombBLAS on NERSC Cori and Summit systems. In the benchmarking experiment shown in Fig. 3, we used the Cori Haswell partition. In the scalability experiments, we use the Cori KNL partition (because the KNL partition has more nodes). To demonstrate the accelerator support described in Section 5 we use the Summit system. Table 3 provides a summary of the systems. On Cori, when the number of thread t is greater than one, we used MPI+OpenMP hybrid parallelization. When t is equal to one, pure MPI is used. In either case, only one thread in every process makes MPI calls.

The properties of the matrices used in evaluation are presented in Table 4. Virus, Archaea, Eukarya, Isolates1, and Isolates2 are protein-similarity networks generated from isolate genomes in the IMG database and are publicly available with HipMCL [38]. Metaclust50 stores similarities of proteins in Metaclust50 (https://metaclust.mmseqs.com/) dataset which contains predicted genes from metagenomes and metatranscriptomes of assembled contigs from IMG/M and NCBI. Friendster, it-2004, arabic-2005, and uk-2002 are various social and road networks available in the SuiteSparse Matrix Collection [39]. Finally, the Hyperlink graph is extracted from the web crawl data [40]. For the experiments demonstrating the evolution of SpGEMM performance in CombBLAS over the years (Section 7.2), we use the matrices Eukarya and Friendster. The experiments demonstrating the scalability of sparse kernels and applications (Section 7.4) are conducted on matrices Eukarya, Metaclust50, and Hyperlink. For the accelerator support demonstration in CombBLAS (Section 7.5), we use HipMCL [38] and PageRank applications. For the former, we use matrices Archaea, Eukarya, and Isolates1, and for the latter we use matrices it-2004, arabic-2005, and uk-2002.

7.2 The Evolution of SpGEMM

SpGEMM is one of the most computationally expensive and widely-used kernels implemented in CombBLAS. The original distributed-memory SpGEMM algorithm was based on 2D Sparse SUMMA with a sequential heap-based local SpGEMM implementation [19]. With the evolution of computational platforms and application requirements, SpGEMM also evolved by adding new algorithms, implementations, and hardware support. First, as supercomputer nodes started to use multicore processors, we implemented multithreaded SpGEMM algorithms. Second, to ensure extreme scalability, we designed and developed new CA SpGEMM algorithms [20]. Third, we implemented new hash-based algorithms that are often faster than the heap-based algorithms [24]. Finally, we develop a batched SpGEMM algorithm that forms output in batches when the entire output does not fit in the aggregate memory of all nodes [36].

Fig. 5 compares the computation and communication costs of different variants of SpGEMM. We observe that using 16 threads per process in 2D SUMMA, the communication cost decreases in both multiplications. The communication reduction is $4\times$ for Friendster and $2\times$ for Isolates. However, multithreading may increase the computation time because the OpenMP parallel codes do not scale linearly with increased thread counts. By contrast, with perfect load balancing, MPI computations are expected to scale linearly. However, with the use of faster processors (e.g., Haswell instead of KNL) and sparse matrices (e.g, Friendster), the communication cost is expected to dominate the computation at extreme scale. Hence, multithreading almost always reduces the overall runtime of our distributed SpGEMM. Next, the hash SpGEMM algorithm reduces the local computational time significantly for both matrices in Fig. 5. The CA SpGEMM further reduces the communication time shown in the last bar. Thus, using

Authorized licensed use limited to: University of Maryland College Park. Downloaded on March 17,2024 at 20:35:50 UTC from IEEE Xplore. Restrictions apply.

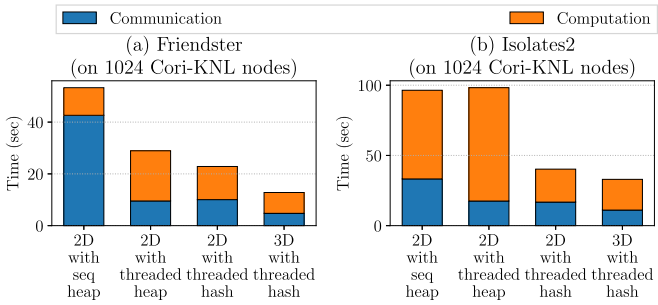


Fig. 5. Distributed SpGEMM Evolution. For the Friendster matrix, multiplication was carried on using 4 batches. 64 cores were used for each node. For multithreaded computations, 4 processes per node and 16 threads per process were used.

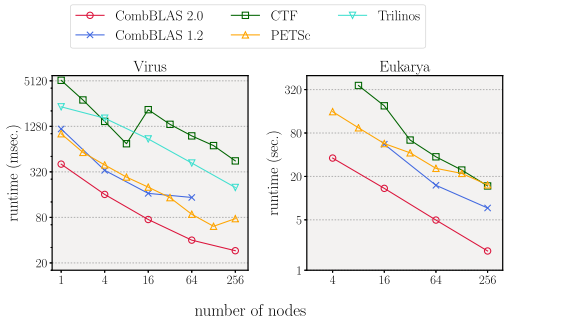


Fig. 6. Parallel SpGEMM runtime of two versions of CombBLAS and other popular parallel sparse linear algebra libraries on two different matrices.

the same computational resources, new algorithms and implementations of SpGEMM in CombBLAS 2.0 run 3× to 5× faster than equivalent algorithms in CombBLAS 1.0. Furthermore, the entire output from squaring the Friendster matrix cannot even be stored on 1024 nodes because of memory limitations (Table 4 shows the output of this multiplication has 1 trillion nonzeros). CombBLAS enables such multiplication by generating results in batches; for example, 4 batches are used in Fig. 5a. This SpGEMM algorithm can be directly used with applications that do not access the whole output matrix all at once, such as many-to-many protein sequence aligner PASTIS [9] and protein clustering algorithm HipMCL [38].

7.3 Comparison Against Other Parallel Sparse Linear Algebra Libraries

We assess the parallel runtime and I/O performance of CombBLAS against three popular parallel sparse linear algebra libraries, CTF [14], PETSc [41], and Trilinos [42]. While these libraries have different focuses, they all support similar parallel sparse matrix operations. We also evaluate CombBLAS 2.0 against the earlier version 1.2. We specifically focus on the performance of SpGEMM and measure its parallel runtime for all evaluated libraries. For these experiments, we use the Virus and Eukarya networks, and run on the Haswell partition of Cori with varying numbers of nodes from 1 to 256. We use 4 MPI processes per node and 8 threads per process for all libraries except CombBLAS 1.2, which did not have multithreaded SpGEMM. For CombBLAS 1.2, we test out various number of MPI tasks per node and report the result of the best run. We plot the obtained parallel runtime results in Fig. 6 and the I/O times in Table 5. Because Trilinos takes too long to read its input matrices, we exclude it from our evaluations for the larger Eukarya network.

Fig. 6 shows that the SpGEMM operation in CombBLAS 2.0 is both more scalable and 3x-4x faster compared to the implementation in CombBLAS 1.2. Additionally, CombBLAS 2.0 is able to run on instances where CombBLAS 1.2 fails due to being out of memory.

CombBLAS 2.0 is faster and more scalable than the three other libraries evaluated for the SpGEMM operation. Among the evaluated libraries, the closest competitor is PETSc, but CombBLAS 2.0 is still 3x-8x faster and more scalable than PETSc. PETSc's performance is similar to that of CombBLAS 1.2, and the other two libraries, CTF and Trilinos trail PETSc and CombBLAS 1.2.

Table 5 shows that CombBLAS and PETSc have faster and more scalable I/O compared to Trilinos and CTF. PETSc requires a custom binary file format for its input, while CTF requires a custom ASCII text format, for which we needed to convert the matrices explicitly. CombBLAS and Trilinos support the common Matrix Market format. In addition, CombBLAS supports a custom binary format, and the I/O performance of CombBLAS in both ASCII and binary formats are fast enough to be of practical use.

7.4 Extreme Scalability of Operations and Applications

Thanks to hybrid parallelization techniques and CA algorithms, CombBLAS 2.0 successfully scales to thousands of nodes on modern supercomputers. Here we demonstrate the strong scaling of CA SpGEMM and a connected component algorithm based on SpMV and SpMSpV.

Fig. 7 shows the strong scaling of CA SpGEMM when squaring a small-scale (Eukarya) and a large-scale (Metaclust50) matrix. Both matrices represent protein similarity networks, and squaring these matrices captures random walks on a graph needed by the protein clustering application HipMCL [38]. Since Eukarya is a small-scale matrix with the runtime dominated by computation, we used just 4 layers in the CA SpGEMM algorithm. For large-scale experiments with Metaclust50, we used 16 layers. When we go from 64 nodes to 1024 nodes (a 16× increase in node counts), CA SpGEMM with Eukarya runs 7× faster. Similarly, when we go from 256 nodes to 4096 nodes (a 16× increase in node counts), CA SpGEMM with Metaclust50 becomes 7× faster. Hence, CA SpGEMM works equally well for both small-scale and large-scale matrices on different concurrencies. More importantly,

TABLE 5
I/O Time (Seconds) of CombBLAS and Various Parallel Sparse Linear Algebra Libraries for Two Networks

Matrix	nodes	CombBLAS		CTF	PETSc	Trilinos
		ASCII	binary			
Virus	4	0.83	0.40	0.75	0.34	145.37
	16	0.41	0.32	0.54	0.31	190.54
	64	0.29	0.48	4.69	0.61	895.33
Eukarya	16	19.41	15.70	43.60	13.33	–
	64	5.31	6.95	26.38	9.91	–
	256	2.31	5.74	26.76	9.52	–

Authorized licensed use limited to: University of Maryland College Park. Downloaded on March 17,2024 at 20:35:50 UTC from IEEE Xplore. Restrictions apply.

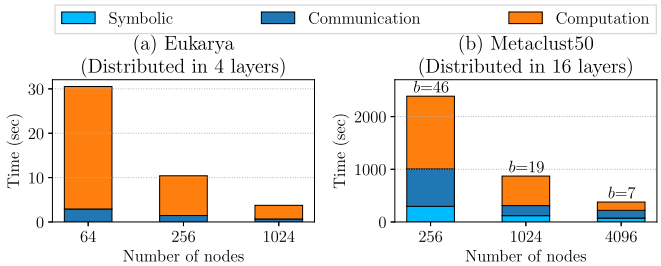


Fig. 7. Strong scaling of CA SpGEMM when squaring (a) Eukarya and (b) Metaclust50. We used 4 MPI processes per node and 16 threads per process on Cori KNL. Number of layers is mentioned at the top of each figure. For Metaclust50, b on top of each bar represents the number of batches needed for SpGEMM (because of the memory limitation). b is calculated using a symbolic step. No batching is needed for Eukarya.

the communication cost continues to reduce with increasing node counts. Thus, given big enough matrices, distributed SpGEMM in CombBLAS 2.0 has the ability to scale to the limit of modern supercomputers.

Fig. 8 shows the strong scaling of FastSV [37], an algorithm for finding connected components in an undirected graph. FastSV is based on the classic Shiloach-Vishkin algorithm [43] and developed using SpMV, SpMSpV and vector assign/extract operations from CombBLAS. Despite relying on matrix-vector multiplications and vector manipulations, FastSV scales remarkably well up to thousands of nodes as shown in Fig. 8. For the relatively small matrix Metaclust50, FastSV does not scale beyond 1024 nodes because of the scaling limitation of SpMSpV. For the Hyperlink graph with 124.9 billion edges, FastSV scales to 4096 nodes (262,144 cores). Since these graphs require more than 1TB memory just to store the input, it is often impossible to analyze them on a typical shared-memory server. Thus, CombBLAS 2.0 expands the scalability horizon of graph analysis, even for algorithms with relatively low computational complexities and irregular communication patterns.

7.5 Benefiting From Accelerators

In this section, we demonstrate two applications powered by CombBLAS partially running on NVIDIA GPUs: High Performance Markov Clustering (HipMCL) optimized for systems containing GPU nodes [33] and the PageRank algorithm. The former makes heavy use of the SpGEMM kernel while the latter mainly relies on SpMV. We use the Summit system for both of these applications. We use one MPI task for each GPU, which results in 6 MPI tasks per node. We use rather modest-sized matrices for these applications – the largest ones containing around 1 billion nonzeros (Table 4).

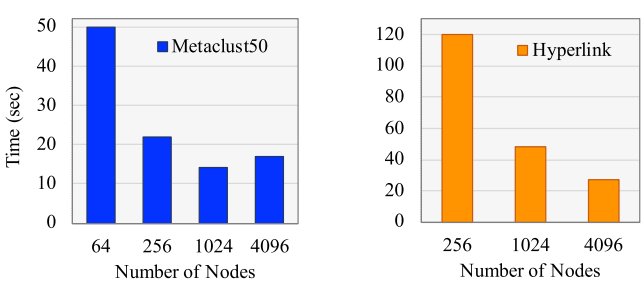
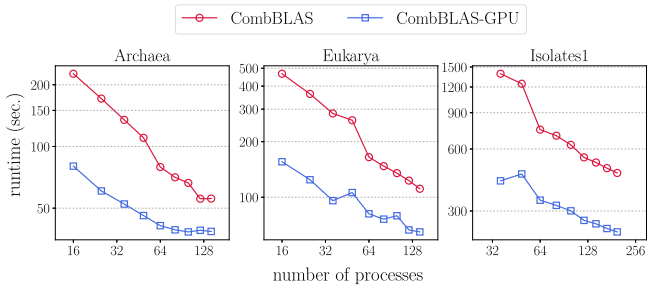
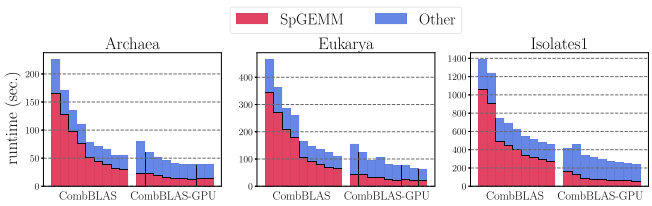


Fig. 8. Scaling FastSV (a distributed-memory connected-component algorithm) on Cori KNL. We used 4 MPI processes per node and 16 threads per process.



(a) Scalability of CombBLAS with and without GPU support.

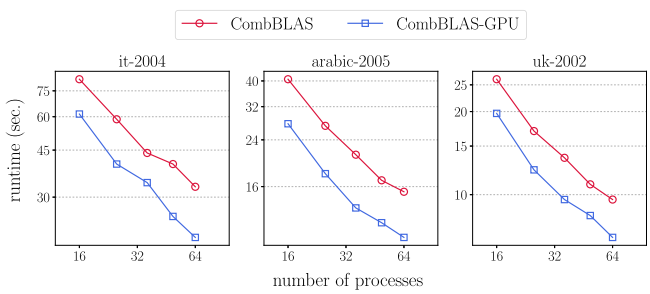


(b) Time spent in SpGEMM.

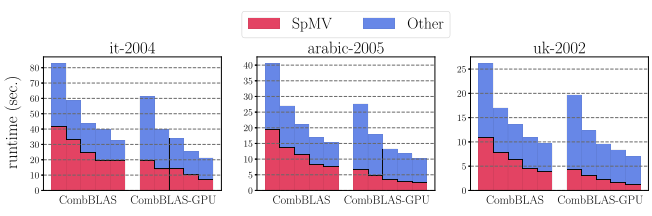
Fig. 9. HipMCL with and without accelerator support. Number of processes $\{p = q^2 : q = 4, 5, \ldots, 12\}$ for Archaea and Eukarya, and $\{p = q^2 : q = 6, 7, \ldots, 14\}$ for Isolates1.

Fig. 9 shows HipMCL running with and without accelerator support. SpGEMM constitutes a very significant portion of overall execution time as seen in Fig. 9b. By offloading these computationally expensive SpGEMM operations to GPUs, CombBLAS is able to obtain $2-3\times$ speedup on all three tested instances. The GPUs are especially effective when the compression ratio of the multiplied matrices is high – which is usually the case for these instances. Note that the baseline CPU-based HipMCL here uses the fastest multithreaded multiplication algorithms presented in this work. GPU support in CombBLAS 2.0 provides more than an order of magnitude speedup [33].

The results for PageRank are presented in Fig. 10. SpMV has lower arithmetic intensity than SpGEMM, and this is reflected in Fig. 10a where the performance improvement



(a) Scalability of CombBLAS with and without GPU support.



(b) Time spent in SpMV.

Fig. 10. PageRank with and without accelerator support. Number of processes $\{p = q^2 : q = 4, 5, \ldots, 8\}$ for all three graphs.

Authorized licensed use limited to: University of Maryland College Park. Downloaded on March 17,2024 at 20:35:50 UTC from IEEE Xplore. Restrictions apply.

due to GPU utilization is around $20-40$ percent, which is smaller than the improvement observed in Fig. 9a. Although we utilize the algorithms recommended by NVIDIA for SpMV in cuSPARSE, the input matrix needs to be transposed due to lack of CSC matrix storage support in cuSPARSE. Despite this slowdown, utilizing GPUs still results in considerable performance gain.

8 Conclusion

CombBLAS is currently the primary backend of several distributed-memory applications such as the graph clustering code HipMCL [38], protein homology search code PASTIS [9], and read-to-read genome aligner diBELLA.2D [10]. Several other applications such as the sparse direct solver SuperLU [44], the structured dense matrix solver STRUMPACK [45], and the sketching library libskylark [46] also utilize CombBLAS for certain tasks.

As evidenced by the experiments shown in this paper, CombBLAS is able to work on sparse matrix operations where the output has 1 trillion nonzeros. Contemporary publications that use CombBLAS also demonstrated that it can successfully utilize 1 million hardware threads [36].

Acknowledgments

The authors would like to thank the assistance of Penporn Koanantakool and Alok Tripathy with some of the figures. The authors would also like to thank Scott Beamer, Saliya Ekanayake, Giulia Guidi, Tristan Konolige, Adam Lugowski, and Alex Reinking, for their contributions on various issues related to CombBLAS development or installation. This work was supported in part by Advanced Scientific Computing Research (ASCR) Program, Office of Science of the DOE, under Grant DE-AC02-05CH11231, in part by National Science Foundation under Grant 1823034, and in part by Exascale Computing Project 17-SC-20-SC, a collaborative effort of the U.S. Department of Energy Office of Science and National Nuclear Security Administration. The work of John Gilbert was supported by NSF under Grant CCF-1637564. This research used resources of Oak Ridge Leadership Computing Facility at Oak Ridge National Laboratory, which was supported by the Office of Science, U.S. Department of Energy, under Contract DE-AC05-00OR22725.

References

[1] J. Kepner *et al.*, "Mathematical foundations of the GraphBLAS," in *Proc. IEEE High Perform. Extreme Comput. Conf.*, 2016, pp. 1–9.
[2] A. Buluç, T. Mattson, S. McMillan, J. Moreira, and C. Yang, "Design of the GraphBLAS API for C," in *Proc. IEEE Int. Parallel Distrib. Process. Symp. Workshops*, 2017, pp. 643–652.
[3] A. Buluç and J. R. Gilbert, "The combinatorial BLAS: Design, implementation, and applications," *Int. J. High Perform. Comput. Appl.*, vol. 25, no. 4, pp. 496–509, May 2011.
[4] W. Gropp, L. N. Olson, and P. Samfass, "Modeling MPI communication performance on SMP nodes: Is it time to retire the ping pong test," in *Proc. 23rd Eur. MPI Users' Group Meeting*, 2016, pp. 41–50.
[5] Y. Saad, *Iterative Methods for Sparse Linear Systems*. Philadelphia, PA, USA: SIAM, 2003.
[6] A. Buluç and J. R. Gilbert, "On the representation and multiplication of hypersparse matrices," in *Proc. IEEE Int. Symp. Parallel Distrib. Process.*, 2008, pp. 1–11.
[7] A. Azad and A. Buluç, "Distributed-memory algorithms for maximum cardinality matching in bipartite graphs," in *Proc. IEEE Int. Parallel Distrib. Process. Symp.*, 2016, pp. 32–42.
[8] A. Azad, M. Jacquelin, A. Buluç, and E. G. Ng, "The reverse Cuthill-McKee algorithm in distributed-memory," in *Proc. IEEE Int. Parallel Distrib. Process. Symp.*, 2017, pp. 22–31.
[9] O. Selvitopi, S. Ekanayake, G. Guidi, G. A. Pavlopoulos, A. Azad, and A. Buluç, "Distributed many-to-many protein sequence alignment using sparse matrices," in *Proc. Int. Conf. High Perform. Comput., Netw., Storage Anal.*, 2020, pp. 1–14.
[10] G. Guidi, O. Selvitopi, M. Ellis, L. Oliker, K. Yelick, and A. Buluç, "Parallel string graph construction and transitive reduction for de novo genome assembly," in *Proc. IEEE Int. Parallel Distrib. Process. Symp.*, 2021, pp. 517–526.
[11] G. Birkhoff and J. D. Lipson, "Heterogeneous algebras," *J. Combinatorial Theory*, vol. 8, no. 1, pp. 115–133, 1970.
[12] T. A. Davis, "Algorithm 1000: SuiteSparse:GraphBLAS: Graph algorithms in the language of sparse linear algebra," *ACM Trans. Math. Softw.*, vol. 45, no. 4, pp. 1–25, 2019.
[13] C. Yang, A. Buluç, and J. D. Owens, "GraphBLAST: A high-performance linear algebra-based graph framework on the GPU," 2019, *arXiv:1908.01407*.
[14] E. Solomonik and T. Hoefler, "Sparse tensor algebra as a parallel programming model," 2015, *arXiv:1512.00066*.
[15] J. Kepner and J. Gilbert, *Graph Algorithms in the Language of Linear Algebra*. Philadelphia, PA, USA: SIAM, 2011.
[16] O. Selvitopi, B. Brock, I. Nisa, A. Tripathy, K. Yelick, and A. Buluç, "Distributed-memory parallel algorithms for sparse times tall-skinny-dense matrix multiplication," in *Proc. ACM Int. Conf. Supercomputing*, 2021, pp. 431–442.
[17] A. Azad, A. Buluç, X. S. Li, X. Wang, and J. Langguth, "A distributed-memory algorithm for computing a heavy-weight perfect matching on bipartite graphs," *SIAM J. Sci. Comput.*, vol. 42, no. 4, pp. C143–C168, 2020.
[18] R. A. van de Geijn and J. Watts, "SUMMA: Scalable universal matrix multiplication algorithm," University of Texas, Austin, TX, USA, Tech. Rep., 1995.
[19] A. Buluç and J. R. Gilbert, "Parallel sparse matrix-matrix multiplication and indexing: Implementation and experiments," *SIAM J. Sci. Comput.*, vol. 34, no. 4, pp. C170–C191, 2012.
[20] A. Azad *et al.*, "Exploiting multiple levels of parallelism in sparse matrix-matrix multiplication," *SIAM J. Sci. Comput.*, vol. 38, no. 6, pp. C624–C651, 2016.
[21] G. Ballard, J. Demmel, O. Holtz, and O. Schwartz, "Minimizing communication in numerical linear algebra," *SIAM J. Matrix Anal. Appl.*, vol. 32, no. 3, pp. 866–901, 2011.
[22] A. Azad and A. Buluç, "A matrix-algebraic formulation of distributed-memory maximal cardinality matching algorithms in bipartite graphs," *Parallel Comput.*, vol. 58, pp. 117–130, 2016.
[23] A. Azad and A. Buluç, "LACC: A linear-algebraic algorithm for finding connected components in distributed memory," in *Proc. IEEE Int. Parallel Distrib. Process. Symp.*, 2019, pp. 2–12.
[24] Y. Nagasaka, S. Matsuoka, A. Azad, and A. Buluç, "Performance optimization, modeling and analysis of sparse matrix-matrix products on multi-core and many-core processors," *Parallel Comput.*, vol. 90, 2019, Art. no. 102545.
[25] A. Azad and A. Buluç, "A work-efficient parallel sparse matrix-sparse vector multiplication algorithm," in *Proc. IEEE Int. Parallel Distrib. Process. Symp.*, 2017, pp. 688–697.
[26] F. G. Gustavson, "Two fast algorithms for sparse matrices: Multiplication and permuted transposition," *ACM Trans. Math. Softw.*, vol. 4, no. 3, pp. 250–269, 1978.
[27] S. Beamer, K. Asanović, and D. Patterson, "Reducing pagerank communication via propagation blocking," in *Proc. IEEE Int. Parallel Distrib. Process. Symp.*, 2017, pp. 820–831.
[28] D. Chakrabarti, Y. Zhan, and C. Faloutsos, "R-MAT: A recursive model for graph mining," in *Proc. SIAM Int. Conf. Data Mining*, 2004, pp. 442–446.
[29] Y. Nagasaka, A. Nukada, and S. Matsuoka, "High-performance and memory-saving sparse general matrix-matrix multiplication for NVIDIA Pascal GPU," in *Proc. 46th Int. Conf. Parallel Process.*, 2017, pp. 101–110.
[30] T. Gale, M. Zaharia, C. Young, and E. Elsen, "Sparse GPU kernels for deep learning," in *Proc. Int. Conf. High Perform. Comput., Netw., Storage Anal.*, 2020, pp. 1–14.

Authorized licensed use limited to: University of Maryland College Park. Downloaded on March 17,2024 at 20:35:50 UTC from IEEE Xplore. Restrictions apply.

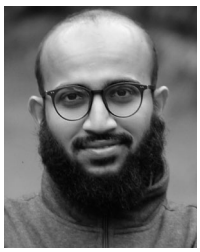
[31] F. Gremse, K. Küpper, and U. Naumann, "Memory-efficient sparse matrix-matrix multiplication by row merging on many-core architectures," *SIAM J. Sci. Comput.*, vol. 40, no. 4, pp. C429–C449, 2018.
[32] W. Liu and B. Vinter, "An efficient GPU general sparse matrix-matrix multiplication for irregular data," in *Proc. IEEE 28th Int. Parallel Distrib. Process. Symp.*, 2014, pp. 370–381.
[33] O. Selvitopi, M. T. Hussain, A. Azad, and A. Buluç, "Optimizing high performance Markov clustering for pre-exascale architectures," in *Proc. IEEE Int. Parallel Distrib. Process. Symp.*, 2020, pp. 116–126.
[34] R. F. Boisvert, R. F. Boisvert, and K. A. Remington, *The Matrix Market Exchange Formats: Initial Design*, vol. 5935. Gaithersburg, MD, USA: National Institute of Standards and Technology, 1996.
[35] S. van Dongen, "Graph clustering by flow simulation," Ph.D. dissertation, Utrecht Univ., Utrecht, The Netherlands, 2000.
[36] M. T. Hussain, O. Selvitopi, A. Buluç, and A. Azad, "Communication-avoiding and memory-constrained sparse matrix-matrix multiplication at extreme scale," in *Proc. IEEE Int. Parallel Distrib. Process. Symp.*, 2021, pp. 90–100.
[37] Y. Zhang, A. Azad, and Z. Hu, "FastSV: A distributed-memory connected component algorithm with fast convergence," in *Proc. SIAM Conf. Parallel Process. Sci. Comput.*, 2020, pp. 46–57.
[38] A. Azad, A. Buluç, G. A. Pavlopoulos, N. C. Kyrpides, and C. A. Ouzounis, "HipMCL: A high-performance parallel implementation of the Markov clustering algorithm for large-scale networks," *Nucleic Acids Res.*, vol. 46, no. 6, Apr. 2018, Art. no. e33.
[39] T. A. Davis and Y. Hu, "The University of Florida sparse matrix collection," *ACM Trans. Math. Softw.*, vol. 38, no. 1, pp. 1–25, Dec. 2011.
[40] R. Meusel, S. Vigna, O. Lehmberg, and C. Bizer, "Graph structure in the web—revisited: A trick of the heavy tail," in *Proc. 23rd Int. Conf. World Wide Web*, 2014, pp. 427–432.
[41] S. Balay *et al.*, "PETSc users manual," Argonne Nat. Lab., Lemont, IL, USA, Tech. Rep. ANL-95/11 - Revision 3.5, 2014.
[42] M. A. Heroux *et al.*, "An overview of the trilinos project," *ACM Trans. Math. Softw.*, vol. 31, no. 3, pp. 397–423, 2005.
[43] Y. Shiloach and U. Vishkin, "An O(log n) parallel connectivity algorithm," *J. Algorithms*, vol. 3, pp. 57–67, 1982.
[44] X. S. Li and J. W. Demmel, "SuperLU_DIST: A scalable distributed-memory sparse direct solver for unsymmetric linear systems," *ACM Trans. Math. Softw.*, vol. 29, no. 2, pp. 110–140, 2003.
[45] F.-H. Rouet, X. S. Li, P. Ghysels, and A. Napov, "A distributed-memory package for dense hierarchically semi-separable matrix computations using randomization," *ACM Trans. Math. Softw. (TOMS)*, vol. 42, no. 4, pp. 1–35, 2016.
[46] G. Kollias *et al.*, "libSkylark: A framework for high-performance matrix sketching for statistical computing," in *Proc. Int. Conf. High Perform. Comput., Netw., Storage Anal. (poster)*, 2015. [Online]. Available: http://sc15.supercomputing.org/sites/all/themes/SC15images/tech_poster/poster_files/post213s2-file3.pdf



Ariful Azad received the PhD degree from Purdue. After that, he was a research scientist with Lawrence Berkeley National Laboratory. He is currently an assistant professor of intelligent systems engineering with Indiana University. His research focuses on high-performance graph analysis and learning.



Oguz Selvitopi received the PhD degree from Bilkent University in 2016. He is currently a research scientist with Lawrence Berkeley National Laboratory. His research interests include high-performance computing, parallel graph algorithms, and bioinformatics.



Md Taufique Hussain is currently working toward the PhD degree with Indiana University Bloomington. His research focuses on high-performance graph algorithms.



John R. Gilbert received the PhD degree from Stanford in 1981. After that, he was with Cornell and Xerox PARC. He is currently a professor of computer science with the University of California, Santa Barbara, and fellow of the Society for Industrial and Applied Mathematics. His research interests include graphs, matrices, and high-performance computing.



Aydın Buluç is currently a staff scientist with Lawrence Berkeley National Laboratory and an adjunct assistant professor of electrical engineering and computer sciences with the University of California, Berkeley. His research interests include parallel computing, combinatorial scientific computing, high-performance graph analysis machine learning, sparse matrix computations, and computational biology.

▷ **For more information on this or any other computing topic, please visit our Digital Library at** www.computer.org/csdl.

Authorized licensed use limited to: University of Maryland College Park. Downloaded on March 17,2024 at 20:35:50 UTC from IEEE Xplore. Restrictions apply.